



Tsinghua University

Modern Computer Architecture

Fall 2025

Lecture 04-1: Multi Issue Introduction

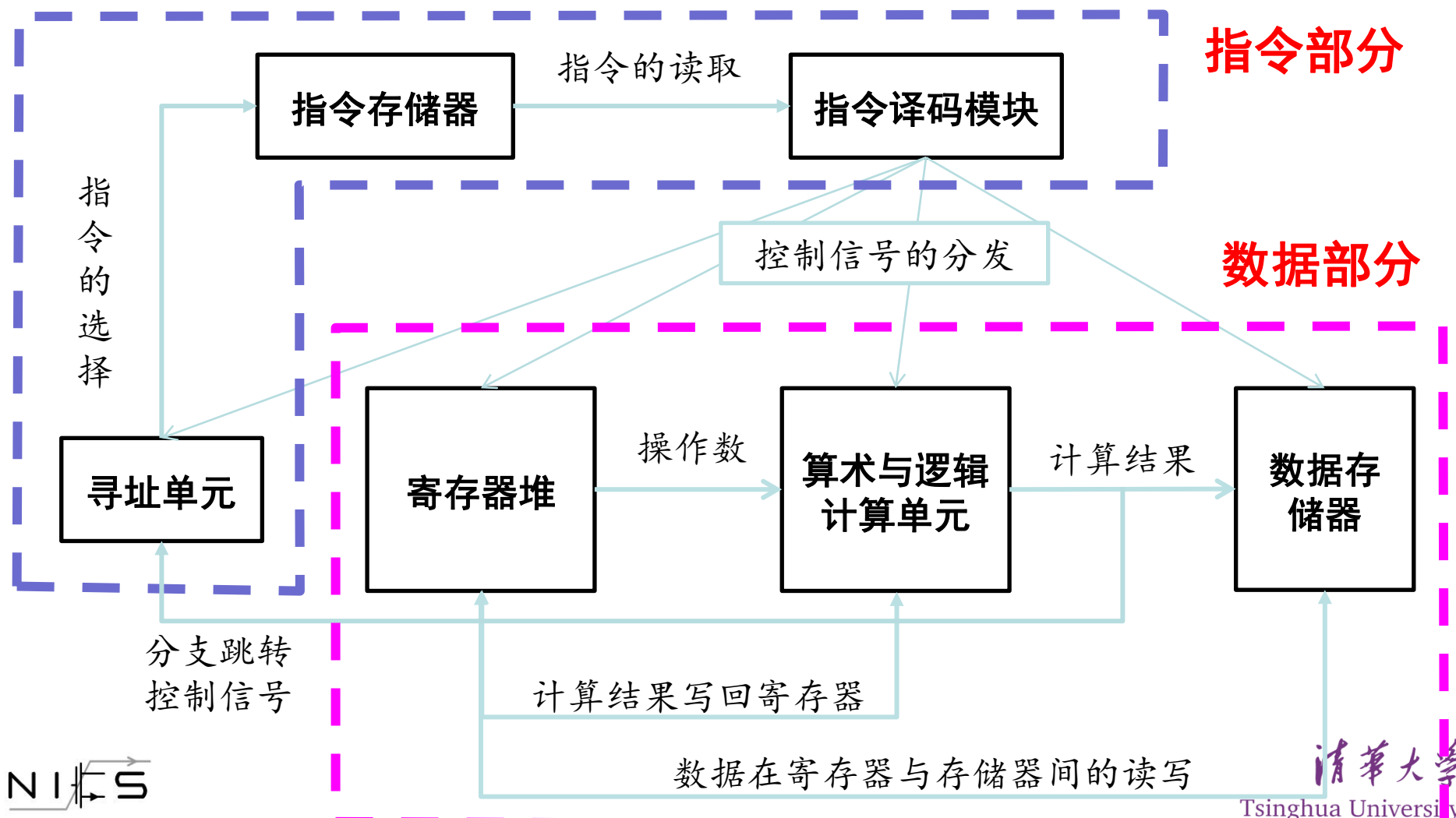
Shuwen Deng

shuwend@tsinghua.edu.cn

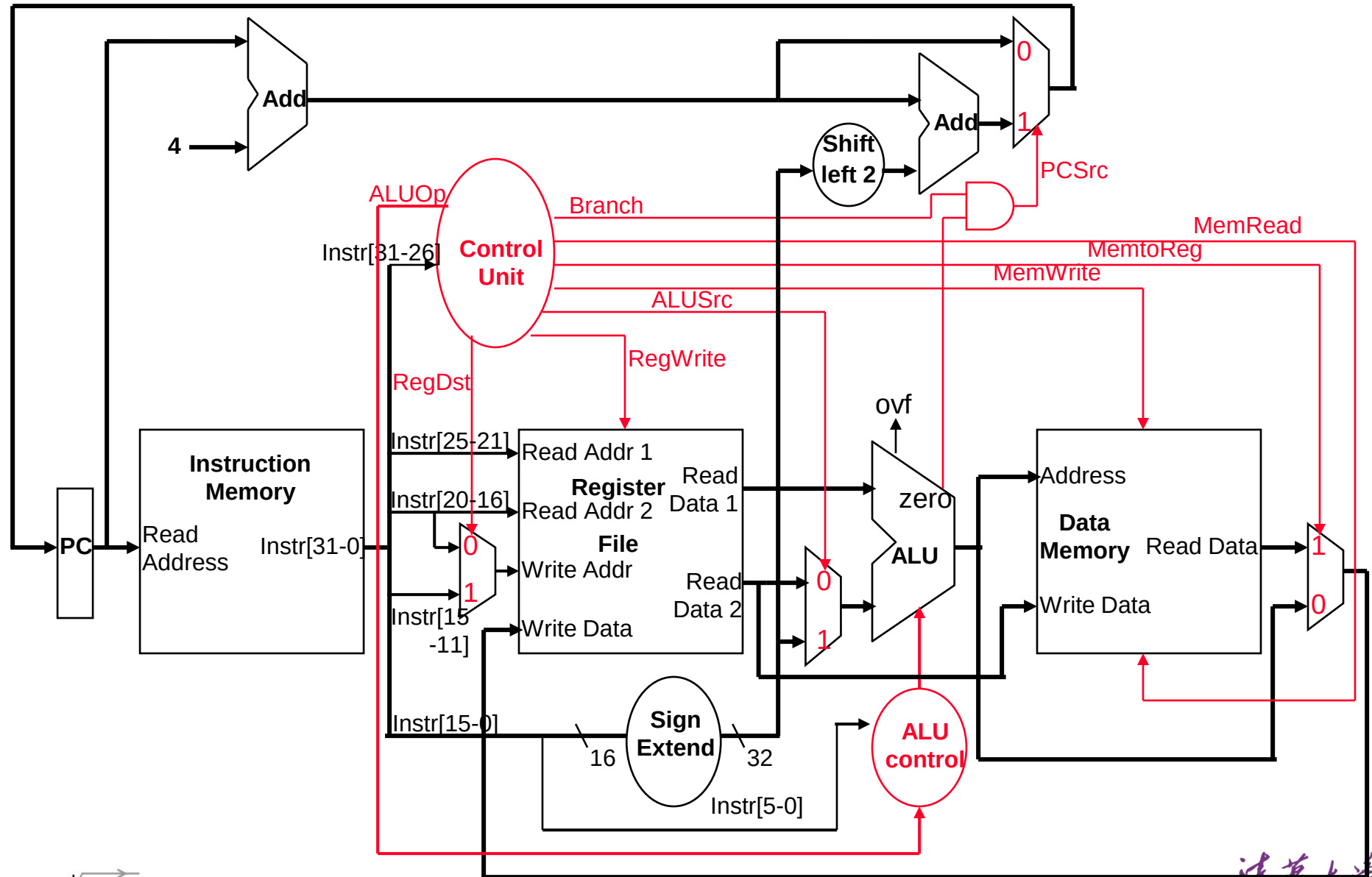
2024-Oct.-15

处理器的架构

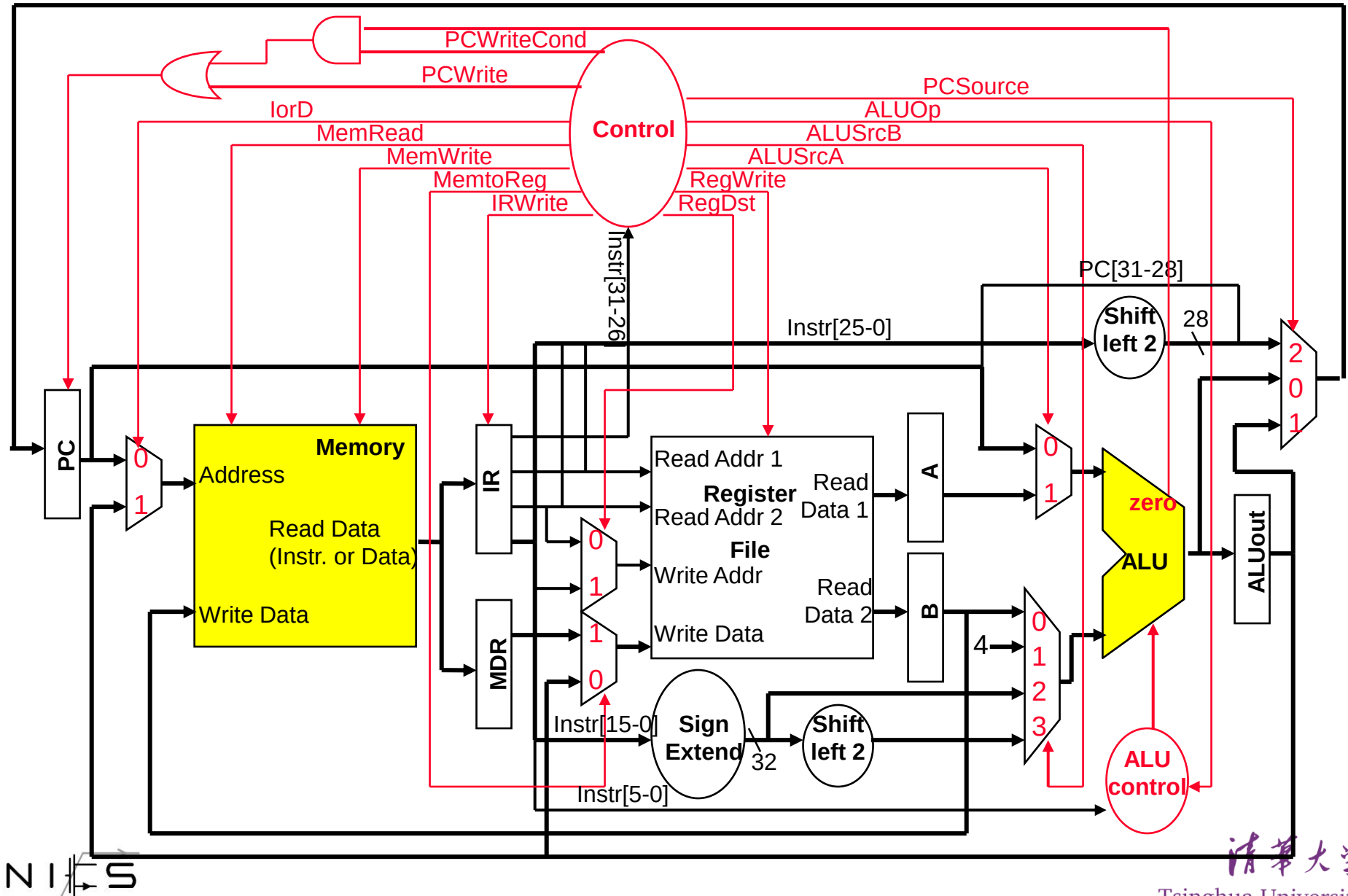
数据通路



Single Cycle Datapath with Control Unit

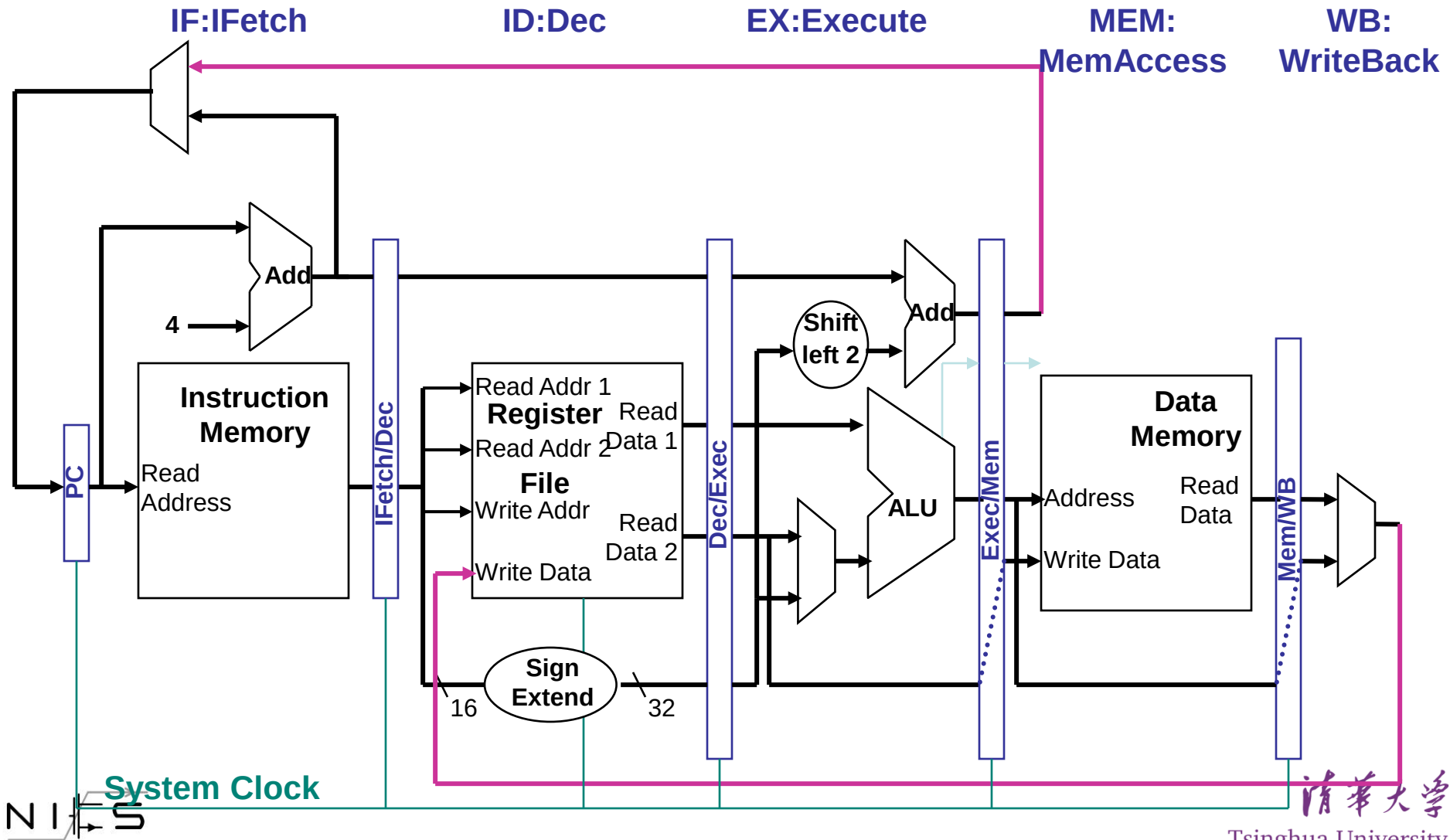


The Multicycle Datapath with Control Signals



MIPS Pipeline Datapath Modifications

- What do we need to add/modify in our MIPS datapath?
 - State registers between each pipeline stage to **isolate** them



Review: Pipeline Hazards

❑ Structural hazards

Design pipeline to eliminate structural hazards

❑ Data hazards – read before write

Use **data forwarding** inside the pipeline

For those cases that forwarding won't solve (e.g., load-use)
include hazard hardware to insert stalls in the instruction stream

❑ Control hazards – `beq, bne, j, jr, jal`

Stall – hurts performance

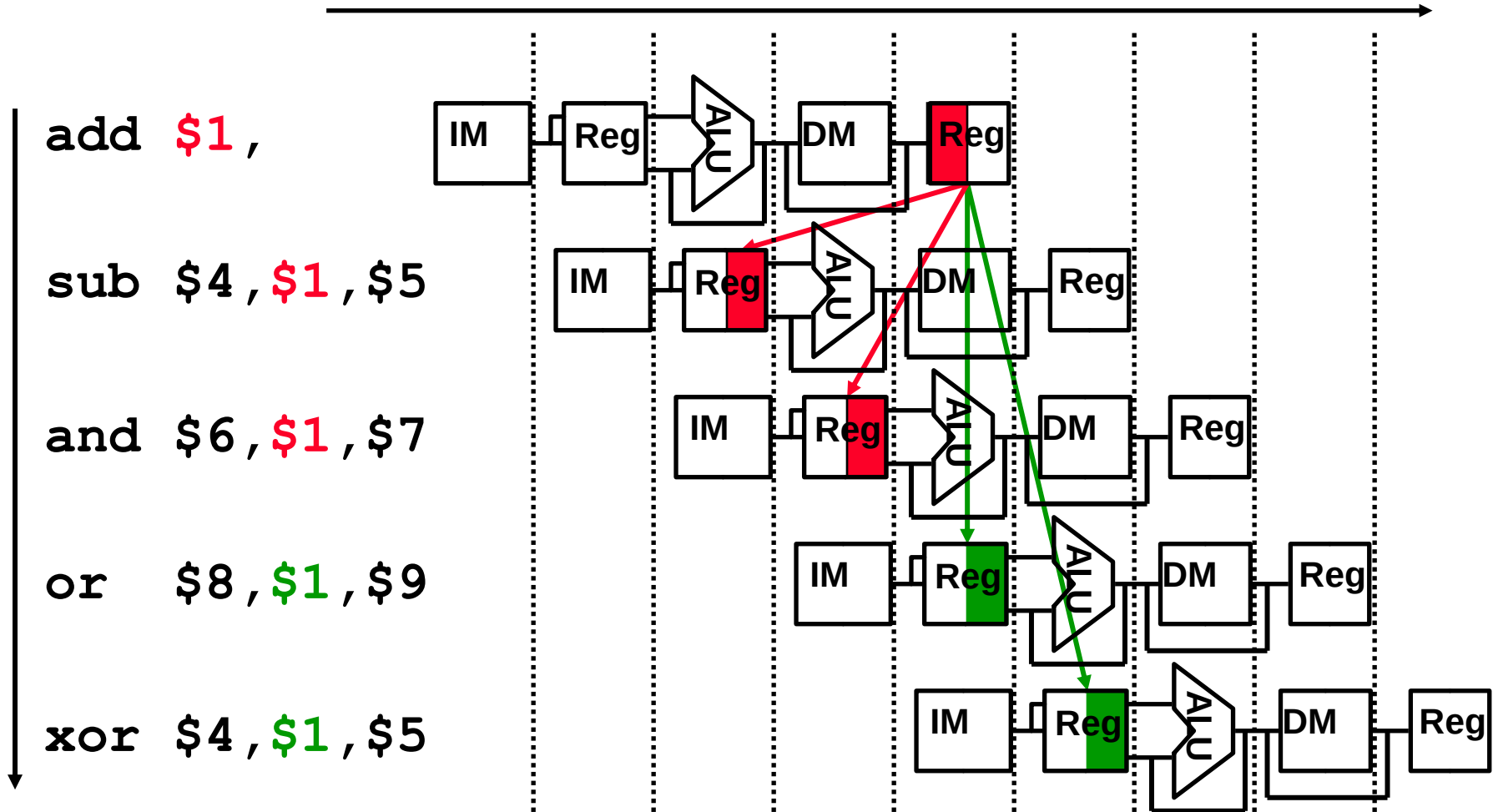
Move decision point as early in the pipeline as possible – reduces number of stalls at the cost of additional hardware

Delay decision (requires compiler support) – not feasible for deeper pipes requiring more than one delay slot to be filled

Predict – with even more hardware, can reduce the impact of control hazard stalls even further if the branch prediction (BHT) is correct and if the branched-to instruction is cached (BTB)

Register Usage Can Cause Data Hazards

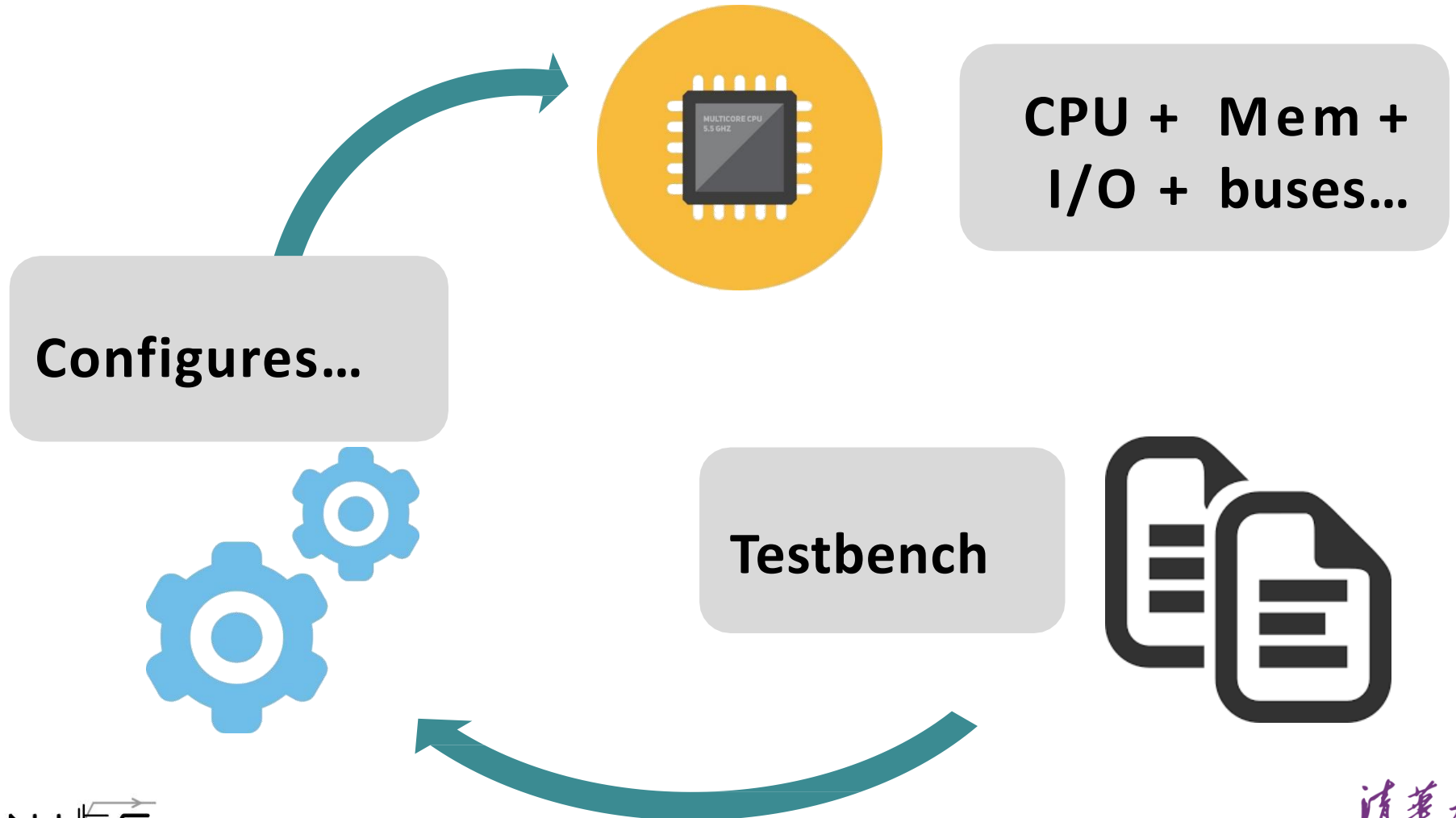
- Dependencies backward in time cause **hazards**



❑ Read before write **data hazard**

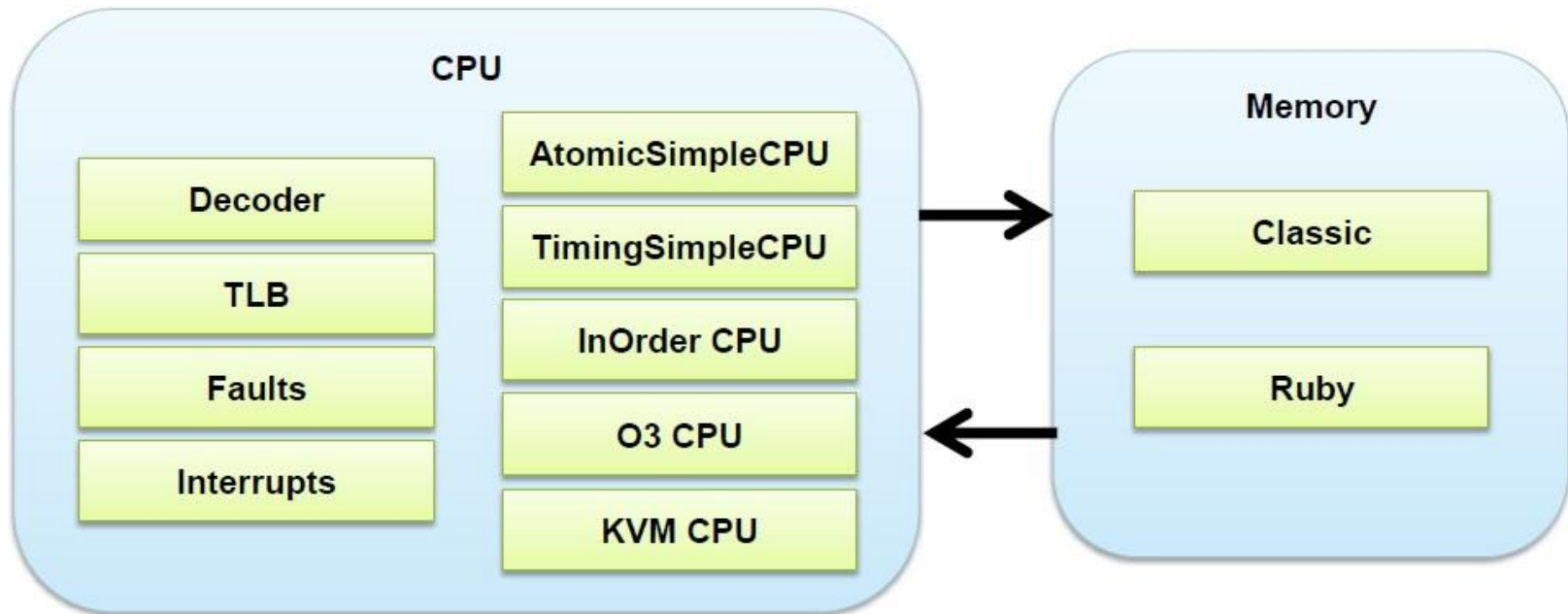
What is gem5???

- GEM5: 在参数配置下，构造计算机系统，运行某程序



Example: CPU Models

- CPU Models are design to be “hot pluggable” with arbitrary ISA and memory systems



Two Fundamental Modes

- Full system (FS)
 - For booting operating systems
 - Models bare hardware, including devices
 - Interrupts, exceptions, privileged instructions, fault handlers
 - Simulated UART output
 - Simulated frame buffer output
- Syscall emulation (SE)
 - For running individual applications, or set of applications on MP
 - Models user-visible ISA plus common system calls
 - System calls emulated, typically by calling host OS
 - Simplified address translation model, no scheduling

After pipeline, What if I want better performance?

- CPU Time = IC $\times$ CPI $\times$ CCT
- The best **CPI** we can achieve so far:
CPI = 1
- The best **CCT** we can achieve so far:
1/5 of that of **single cycle processor**

What if we want better performance?

Extracting Yet *More* Performance

- Two options:
 - Lower CCT / Higher CR :: **superpipelining**
(a brief intro today)
 - Increase the depth of the pipeline to increase the clock rate
 - Lower CPI / Higher IPC :: **multiple-issue** :: parallel
(Main Topic of the following three week)
 - Fetch (and execute) more than one instructions at one time
(expand every pipeline stage to accommodate multiple instructions)

$$\text{CPU Time} = IC \times CPI \times CCT$$

Extracting Yet *More* Performance

- Launching multiple instructions per stage allows the instruction execution rate, CPI, to be less than 1
 - we use **IPC**: instructions per clock cycle
 - 6 GHz, four-way multiple-issue processor
 - 2.4 billion instructions per second
 - $\text{CPI} = 0.25$ or $\text{IPC} = 4$
 - Five-stage pipeline, how many instructions are active in the pipeline at any given time?
 - 5 -> ?

Instruction v.s. Machine Parallelism

- **Instruction-level parallelism (ILP)** of a program – a measure of the average number of instructions in a program that a processor *might* be able to execute at the same time
 - Mostly determined by the number of true (data) dependencies and procedural (control) dependencies in relation to the number of other instructions

ILP: Instruction-Level Parallelism

- Compare the following two pieces of code and Consider data dependency:

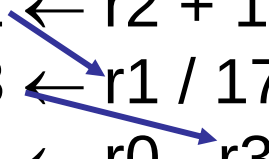
code1:	$r1 \leftarrow r2 + 1$
	$r3 \leftarrow r1 / 17$
	$r4 \leftarrow r0 - r3$

code2:	$r1 \leftarrow r2 + 1$
	$r3 \leftarrow r9 / 17$
	$r4 \leftarrow r0 - r10$

ILP: Instruction-Level Parallelism

- Compare the following two pieces of code and Consider data dependency:

code1: $r1 \leftarrow r2 + 1$
 $r3 \leftarrow r1 / 17$
 $r4 \leftarrow r0 - r3$

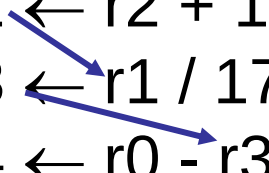


code2: $r1 \leftarrow r2 + 1$
 $r3 \leftarrow r9 / 17$
 $r4 \leftarrow r0 - r10$

ILP: Instruction-Level Parallelism

- Compare the following two pieces of code and Consider data dependency:

code1: $r1 \leftarrow r2 + 1$
 $r3 \leftarrow r1 / 17$
 $r4 \leftarrow r0 - r3$



code2: $r1 \leftarrow r2 + 1$
 $r3 \leftarrow r9 / 17$
 $r4 \leftarrow r0 - r10$

- ILP is a measure of the amount of inter-dependencies between instructions

$$\text{Average ILP} = \frac{\# \text{ of Instruction}}{\# \text{ of Cycle Required}}$$

code1: ILP = 1

i.e. must execute serially

code2: ILP = 3

i.e. can execute at the same time

Instruction v.s. Machine Parallelism

- **Instruction-level parallelism (ILP)** of a program – a measure of the average number of instructions in a program that a processor *might* be able to execute at the same time
 - Mostly determined by the number of true (data) dependencies and procedural (control) dependencies in relation to the number of other instructions
- **Machine-level parallelism** of a processor – a measure of the ability of the processor to take advantage of the ILP of the program
 - Determined by the number of instructions that can be fetched and executed at the same time
- To achieve high performance, need **both** ILP and machine level parallelism

Multiple-Issue Processor Styles

- Static multiple-issue processors
(aka **Very Long Instruction Word, VLIW**)
 - Decisions on which instructions to execute simultaneously are being made **statically** (at ? time by the ?)
 - E.g. Intel Itanium and Itanium 2 for the IA-64 ISA-EPIC (Explicit Parallel Instruction Computer), and TI TMS320C6x
- Dynamic multiple-issue processors (aka **SuperScalar, SS**)
 - Decisions on which instructions to execute simultaneously are being made **dynamically** (at ? time by the ?)
 - E.g. IBM Power 2, Pentium 4, MIPS R10K, HP PA 8500

Multiple-Issue Processor Styles

- Static multiple-issue processors
(aka **Very Long Instruction Word, VLIW**)
 - Decisions on which instructions to execute simultaneously are being made **statically** (at compile time by the Compiler)
 - E.g. Intel Itanium and Itanium 2 for the IA-64 ISA-EPIC (Explicit Parallel Instruction Computer), and TI TMS320C6x
- Dynamic multiple-issue processors (aka **SuperScalar, SS**)
 - Decisions on which instructions to execute simultaneously are being made **dynamically** (at run time by the Hardware)
 - E.g. IBM Power 2, Pentium 4, MIPS R10K, HP PA 8500

OUTLINE of the following three week

- Intro (Week 4)
 - superpipeline & multi-issue
 - Dependency, Hazard,
- SuperPipeline (Week 4)
 - A very brief introduction
- Multi-issue
 - VLIW (Week 4) “static”
 - SuperScalar “dynamic”
 - Basic Model (Week 5)
 - RUU & LSQ (Week 5) -> Scheduling Strategy
 - FrontEnd & BackEnd (Week 6)-> DataPath

SuperPipelined Processors

- Increase the depth of the pipeline leading to shorter clock cycles (and more instructions “in flight” at one time)

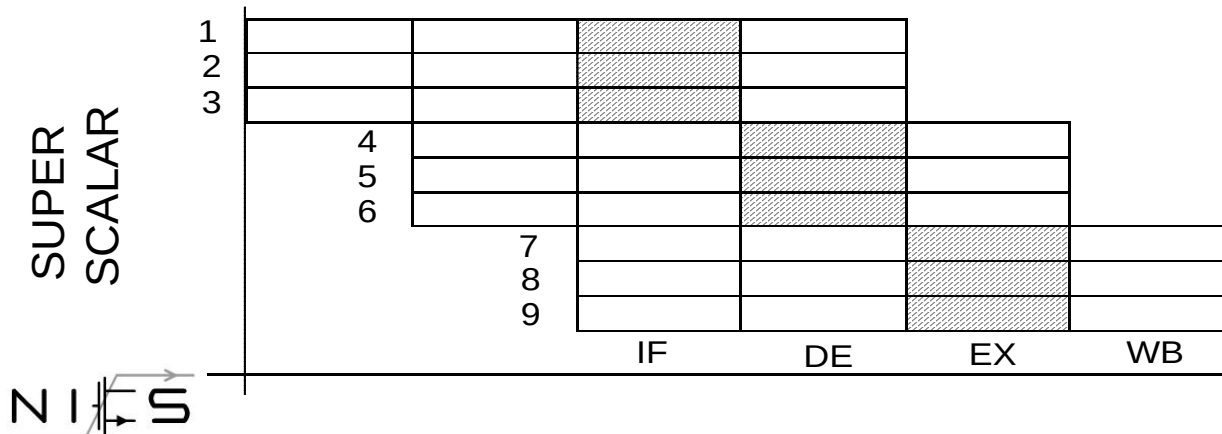
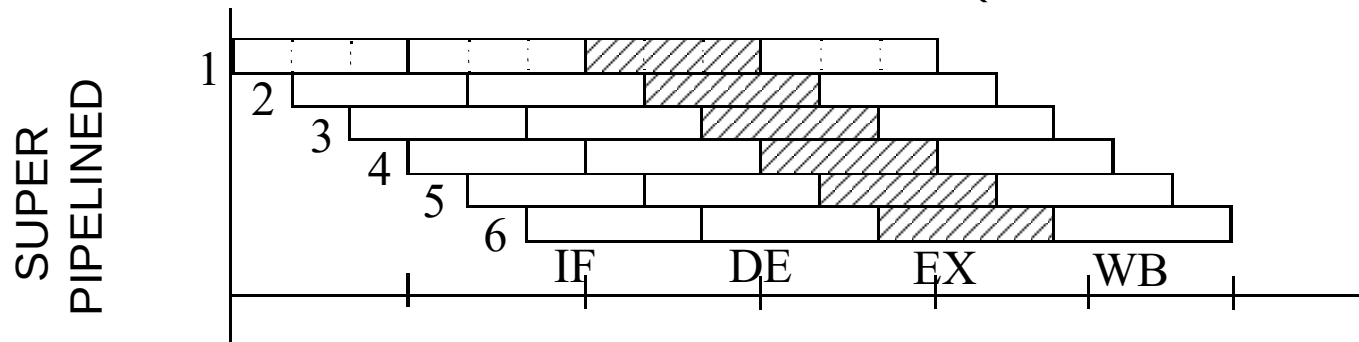
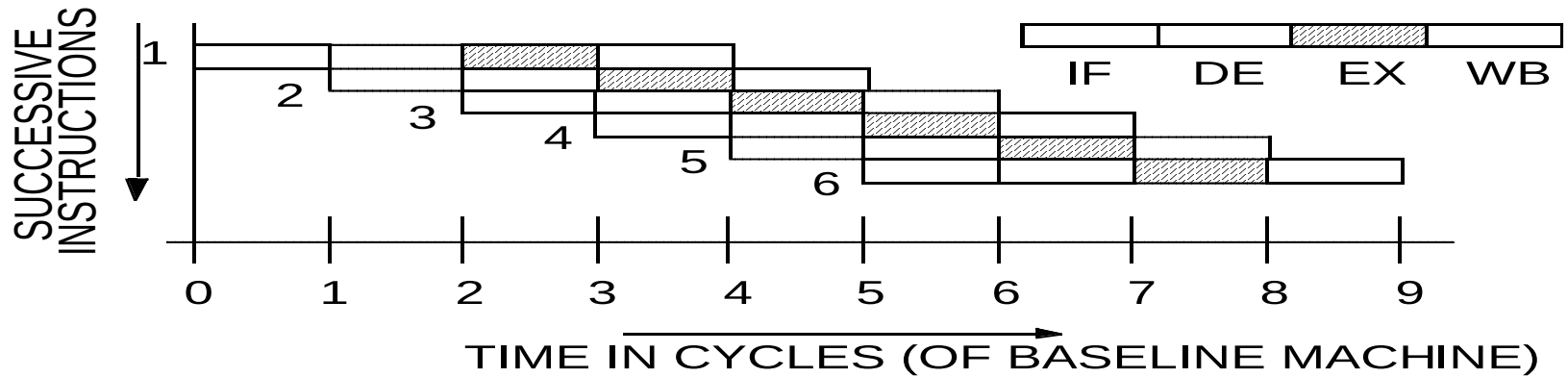
The higher the degree of superpipelining:

- the **more forwarding/hazard hardware** needed,
- the more **pipeline latch overhead** (setup time & hold time, which accounts for a larger percentage of the clock cycle time),
- the bigger **the clock skew issues** (because of faster clocks)

Superpipelined vs Superscalar

- **Superpipelined** processors have **longer instruction latency** than the SS processors, which can degrade performance in the presence of true dependencies
- **Superscalar** processors are more susceptible to resource conflicts – but we can fix this with hardware !

Superpipelined vs. Superscalar [Jouppi, DECWRL 1991]



Multiple-Issue Datapath Responsibilities

- Must handle, with a combination of hardware and software fixes, the fundamental limitations of
 - **Data dependencies** – aka data hazards
 - Limitation more severe in a SS/VLIW processor due to (usually) low ILP
 - **Procedural dependencies** – aka control hazards
 - Ditto, but even more severe
 - Use dynamic branch prediction to help resolve the ILP issue
 - **Resource conflicts** – aka structure hazards
 - A SS/VLIW processor has a much larger number of potential resource conflicts
 - Functional units may have to arbitrate for result buses and register-file write ports
 - Resource conflicts can be eliminated by **duplicating the resource** or by **pipelining the resource**

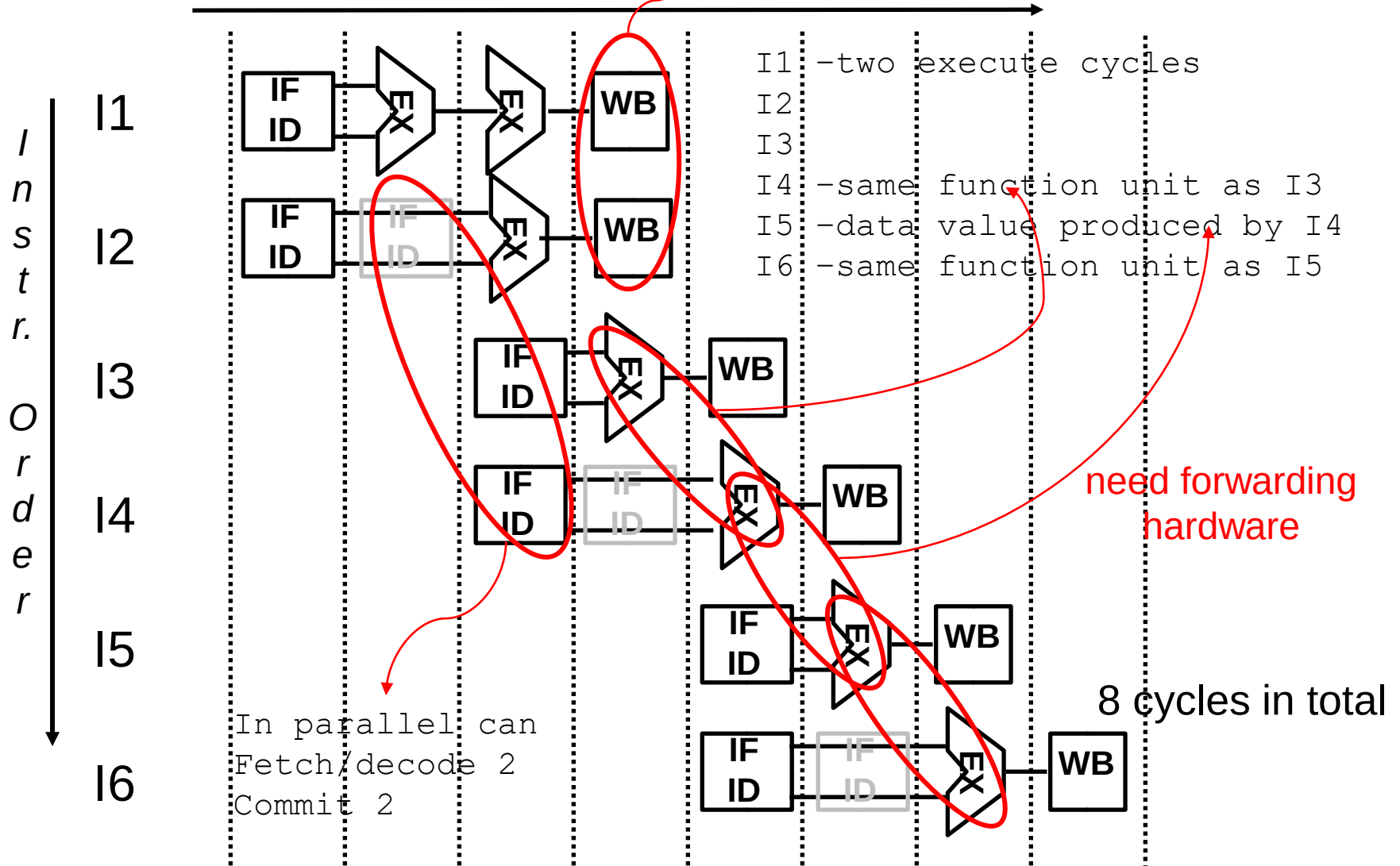
Instruction Issue and Completion Policies

- **Instruction-issue** – initiate execution
 - **Instruction lookahead** capability – ability of the processor to fetch, decode and issue instructions beyond the current point of execution to try to find more instructions to issue
 - **Instruction-completion** – complete execution
 - **Processor lookahead** capability – ability of the processor to examine issued instructions beyond the current point of execution to try to find more instructions to complete
- In-order issue with in-order completion
- In-order issue with out-of-order completion
- Out-of-order issue with out-of-order completion

In-Order Issue with In-Order Completion

- Simplest policy is to issue instructions in exact program order and to complete them in the same order they were fetched (i.e., in program order)
- IOI-IOC Example:
 - Fetch and decode **two** instructions per cycle,
 - **Three** functional units (a single cycle adder, a single cycle shifter, and a two cycle multiplier),
 - Write back **two** results per cycle
 - An instruction sequence with the following characteristics
 - I1 - needs two execute cycles (a multiply)
 - I2
 - I3
 - I4 - needs the same function unit as I3
 - I5 - needs data value produced by I4
 - I6 - needs the same function unit as I5

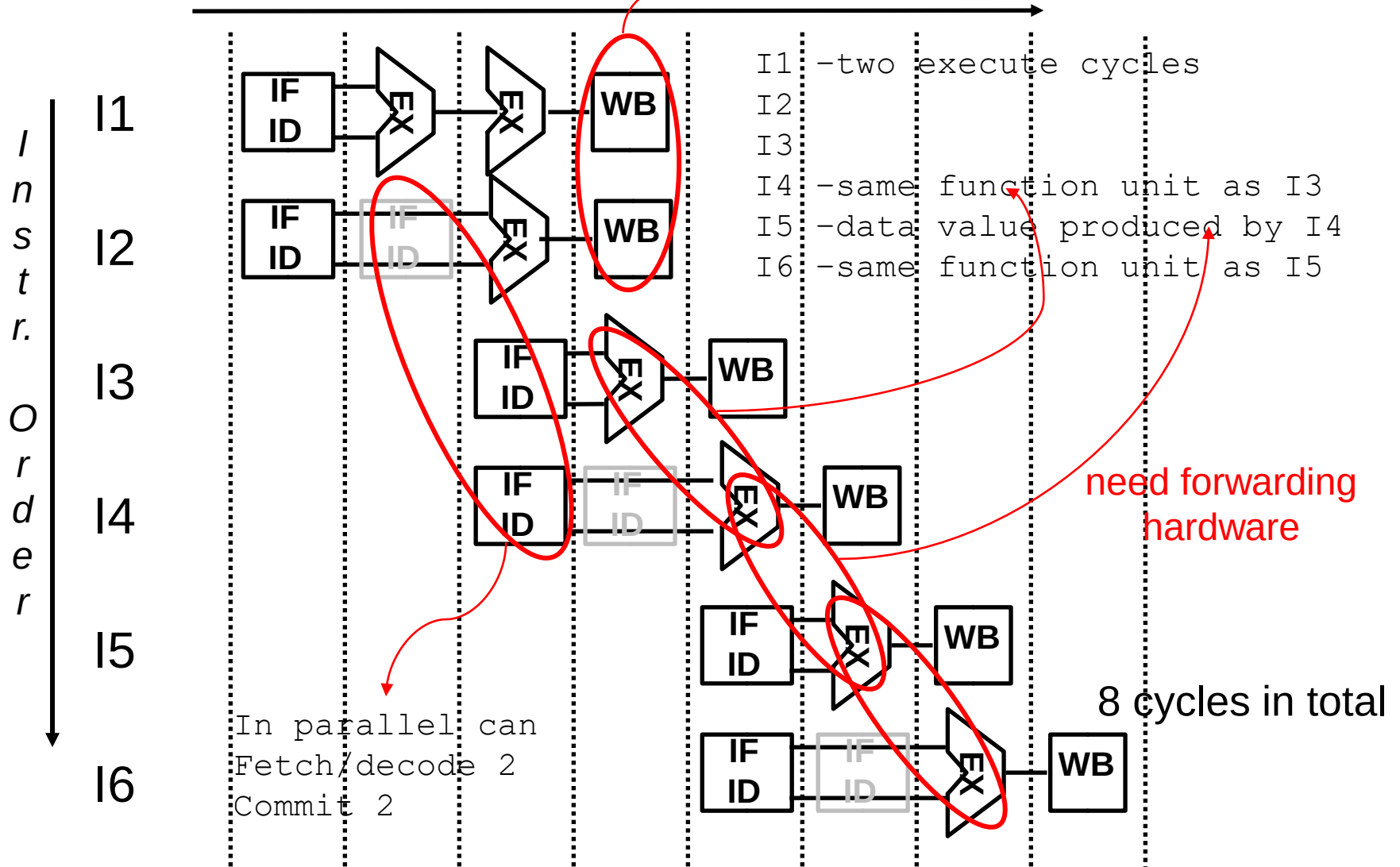
IOI-IOC Example



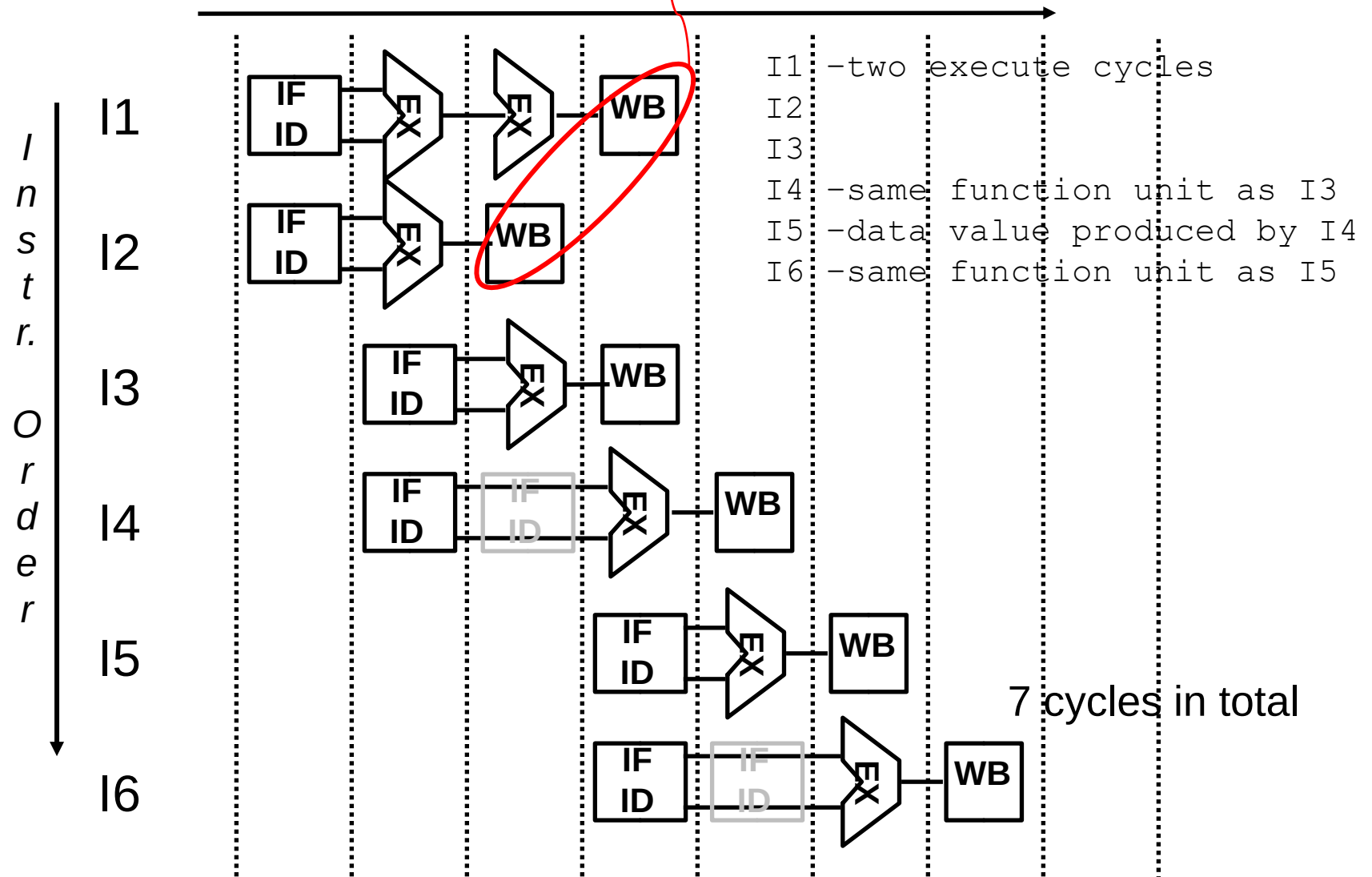
In-Order Issue with Out-of-Order Completion

- With out-of-order completion, a later instruction may complete **before** a previous instruction
 - Out-of-order completion is used in pipelined processors to improve the performance of long-latency operations such as divide
- When using out-of-order completion, instruction issue is **stalled** when there is a resource conflict (e.g., for a functional unit) or when the instructions ready to issue need a result that has not yet been computed

IOI-IOC to IOI-OOC



IOI-OOC Example



Handling Output Dependencies

- There is one more situation that stalls instruction issuing with IOI-OOC, assume
 - I1 – writes to R3
 - I2 – writes to R3
 - I5 – reads R3

If the I1 write occurs **after** the I2 write, then I5 reads an incorrect value for R3

I2 has an **output dependency** on I1 – **write before write**

- The issuing of I2 would have to be stalled if its result might later be overwritten by an previous instruction (i.e., I1) that takes longer to complete – the stall happens before **instruction issue**

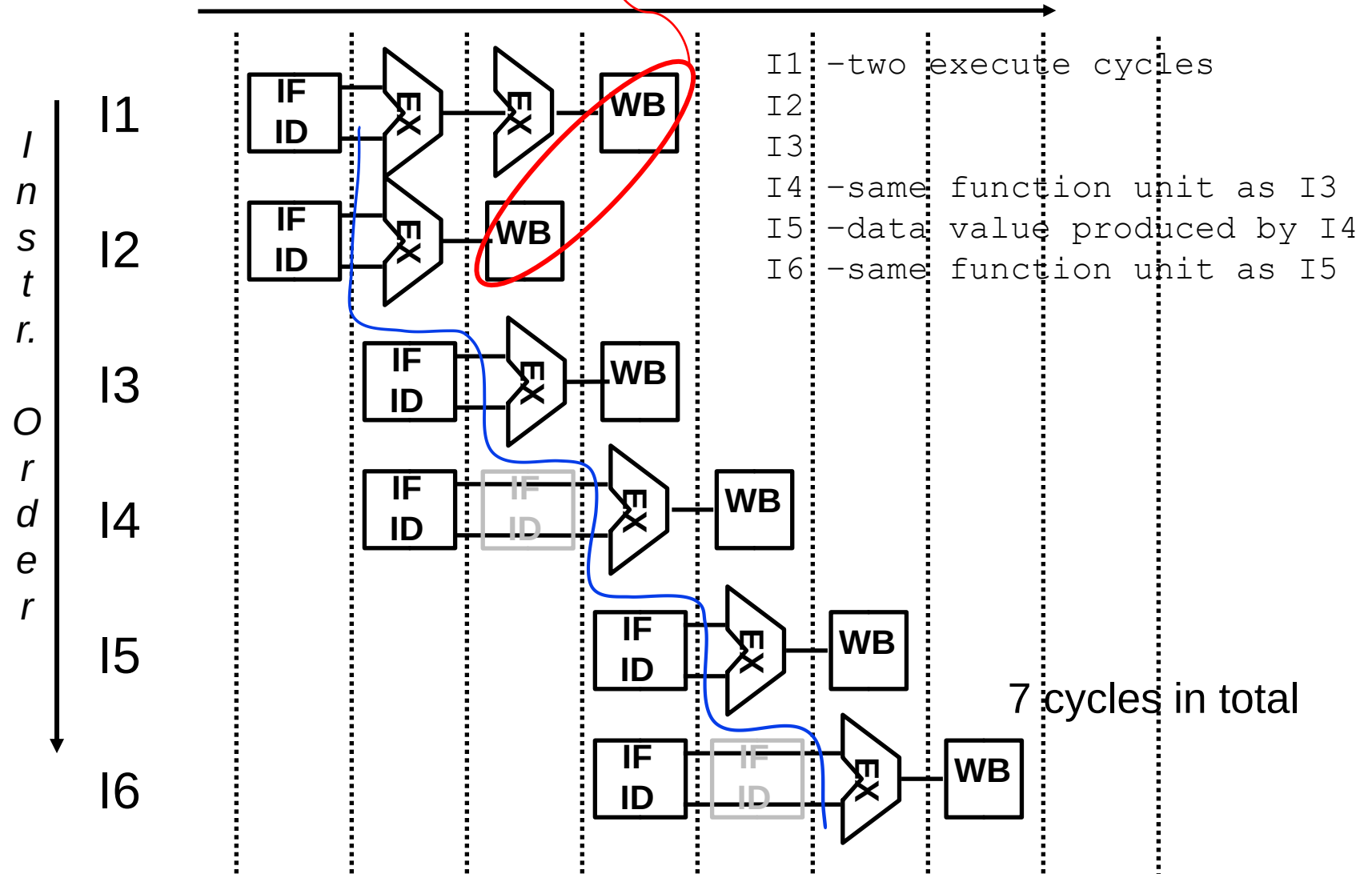
- While IOI-OOC yields higher performance, it requires more dependency checking hardware

Dependency checking needed to resolve both **read before write** and **write before write**

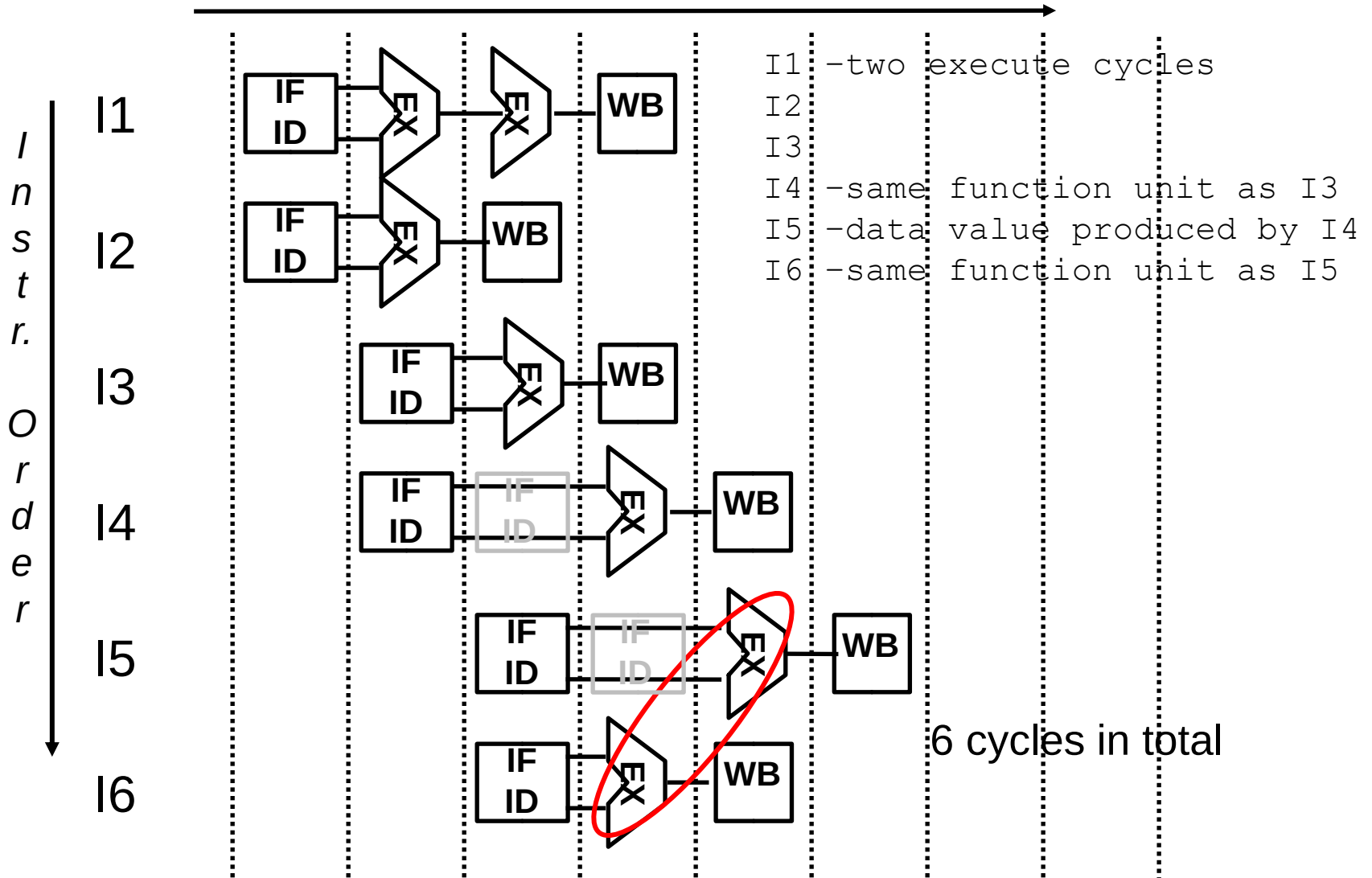
Out-of-Order Issue with Out-of-Order Completion

- **With in-order issue the processor stops** decoding instructions whenever a decoded instruction has a resource conflict or a data dependency on an issued, but uncompleted instruction
 - The processor is not able to *look beyond* the conflicted instruction even though more downstream instructions might have no conflicts and thus be issuable
- Can overcome this limitation by inserting a buffer between the decode and execute stages
 - **instruction buffer** or **instruction window**
 - Fetch and decode instructions *beyond* the conflicted one, store them in an **instruction buffer** (as long as there's room), and flag those instructions in the buffer that don't have resource conflicts or data dependencies
 - Flagged instructions are then issued from the buffer without regard to their program order

IOI-OOO to OOI-OOO



OOI-OOC Example



Antidependencies

- With OOI *also* have to deal with data **antidependencies**
 - when a later instruction (that completes earlier) produces a data value that destroys a data value used as a source in an earlier instruction (that issues later)

$\textcircled{R3} := R3 * R5$
 $R4 := R3 + 1$
 $\textcircled{R3} := R5 + 1$

True data dependency

Output dependency

Antidependency

- The constraint is similar to that of true data dependencies, except *reversed*

True data dependency: The later instruction uses a value (not yet) produced by an earlier instruction (**read before write**);

Anti dependency: The later instruction destroys a value that the earlier instruction (has not yet) used (**write before read**).

Dependencies Review

- Each of the three data dependencies
 - True data dependencies (read before write)
 - Antidependencies (write before read)
 - Output dependencies (write before write)
- } storage conflicts

manifests itself through the use of registers (or other storage locations)

- **True dependencies** represent the flow of data and information through a program
- **Anti- and output dependencies** arise because the limited number of registers mean that programmers **reuse registers** for different computations. When instructions are issued out-of-order, the correspondence between registers and values breaks down and the values *conflict* for registers

Exercise

Example The following loop has multiple types of dependences. Find all the true dependences, output dependences, and antidependences, and eliminate the output dependences and antidependences by renaming.

```
for (i=0; i<100; i=i+1) {  
    Y[i] = X[i] / c; /* S1 */  
    X[i] = X[i] + c; /* S2 */  
    Z[i] = Y[i] + c; /* S3 */  
    Y[i] = c - Y[i]; /* S4 */  
}
```

- True data dependencies (read before write)
- Antidependencies (write before read)
- Output dependencies (write before write)

Exercise

Example The following loop has multiple types of dependences. Find all the true dependences, output dependences, and antidependences, and eliminate the output dependences and antidependences by renaming.

```
for (i=0; i<100; i=i+1) {  
    Y[i] = X[i] / c; /* S1 */  
    X[i] = X[i] + c; /* S2 */  
    Z[i] = Y[i] + c; /* S3 */  
    Y[i] = c - Y[i]; /* S4 */  
}
```

Answer The following dependences exist among the four statements:

1. There are true dependences from S1 to S3 and from S1 to S4 because of Y[i]. These are not loop carried, so they do not prevent the loop from being considered parallel. These dependences will force S3 and S4 to wait for S1 to complete.
2. There is an antidependence from S1 to S2, based on X[i].
3. There is an antidependence from S3 to S4 for Y[i].
4. There is an output dependence from S1 to S4, based on Y[i].

Storage Conflicts Solution -- Register Renaming

- Storage conflicts can be reduced (or eliminated) by increasing or duplicating the troublesome resource
 - Provide additional registers that are used to reestablish the correspondence between registers and values
 - Allocated dynamically by the hardware in SS processors
- Register Renaming – the processor renames the original register identifier in the instruction to a new register (one not in the visible register set)

$\text{R3} := \text{R3} * \text{R5}$	$\Rightarrow$	$\text{R3b} := \text{R3a} * \text{R5a}$
$\text{R4} := \text{R3} + 1$		$\text{R4a} := \text{R3b} + 1$
$\text{R3} := \text{R5} + 1$		$\text{R3c} := \text{R5a} + 1$

The hardware that does renaming assigns a “replacement” register from **a pool of free registers** and releases it back to the pool when its value is superseded and there are no outstanding references to it

Summary

- Parallelism
 - Instruction-Level & Machine-Level
 - SuperPipeline
 - Multi-issue
 - SuperScalar: Hardware
 - Very Long Instruction Word : Compiler
 - Problem : Data Dependency
 - True data dependency : IOI-IOC (RaW)
 - Output dependency : IOI-OOC (WaW)
 - Anti dependency : OOI-OOC (WaR)